

CSCI 4372/5372/6397: Data Clustering

Phase 4: Internal Validation

Submission Deadline: **Wednesday, April 1 (23:59)**

Objective: Implement internal validation methods to determine the number of clusters in a given data set automatically.

Perhaps the most significant drawback of k -means is that it requires the user to supply the number of clusters (K). In many applications, this is impossible or impractical.

In this phase, you will implement two internal validation methods that will help k -means automatically determine the number of clusters. An internal validation method quantifies the match between an automatically generated partition of a data set and the data set itself. Internal validation is often performed using an internal validity index, a function that takes a partition, the data set itself, and possibly additional parameters as input and returns a numerical value indicating the quality of the partition.

In this phase, you will implement the Calinski–Harabasz (CH) and Silhouette Width (SW) internal validity indices. These indices are described in many resources, see for example, the following recent book by Zaki and Meira Jr.: https://dataminingbook.info/book_html/

Bonus for 4372 Students [10 points]; Mandatory for 5372 and 6397 Students: In addition to the CH and SW indices, implement the Dunn (D) index (for a description, refer to the aforementioned book).

Bonus for 4372 and 5372 Students [10 points]; Mandatory for 6397 Students: In addition to the CH, SW, and D indices, implement the Davies–Bouldin (DB) index (for a description, refer to the aforementioned book).

CH, SW, and D are maximization indices, whereas DB is a minimization index. The way these indices are used is quite simple. For a given data set, we first decide $K_{\min}$ and $K_{\max}$, the minimum and maximum number of clusters that might be present in the data set, respectively. For example, for the iris data set, $K_{\min}$ and $K_{\max}$ could be 2 and 9, respectively. For any data set, $K_{\min}$ is typically 2, whereas, a rule of thumb on the maximum possible $K_{\max}$ value is the nearest integer to $\sqrt{N/2}$, where N is the number points in the data set. Assuming that we have a randomized initialization method, we then run k -means R times for $K = 2$ and compute the internal validity index (say, CH) on the partition generated in the best run (that is, the run that produced the smallest SSE). We do the same for $K = 3, 4, \dots, 8$, and, finally, $K = 9$. Since CH is a maximization criterion, we then find the K value that produced the largest CH value. That K value is the “estimated” number of clusters in the data set. For a numerical example, refer to the example given below and to Example 17.8 in the aforementioned book. Note that, these indices

all give *estimates*; so, you may not find the “correct” (3) number of clusters for iris (or, for any other real-world data set).

Numerical Example for SW: SW values for the iris data set (**only** for $K = 2$ and 3 clusters—in your experiments, the $K_{\max}$ value for this data set should be $\text{round}\{\sqrt{150/2}\} = 9$) are given below. Note that this is just to give you an example of how reasonable SW values should look like; you may **not** get the exact same numbers depending on your initialization. In addition, these numbers were obtained on the raw (unnormalized) iris data set. In your experiments, you **must** normalize the data sets using the min-max method (Phase 3). There are 3 real clusters in iris, but two of those clusters overlap, so $K = 2$ gives a much better SW value than $K = 3$. So, if we were to try just $K = 2$ and 3 clusters, based on the SW index, we would select $K = 2$ as the best estimate for the number of clusters in this particular data set. Remember that if you are getting silhouette values outside the $[-1, 1]$ range, there **must** be a bug in your program.

```
./test data/iris_bezdek.txt 2
Iteration 1: SSE = 539.413
Iteration 2: SSE = 366.075
Iteration 3: SSE = 237.899
Iteration 4: SSE = 175.767
Iteration 5: SSE = 154.339
Iteration 6: SSE = 152.513
Iteration 7: SSE = 152.348
SW(2) = 0.850351
```

```
./test data/iris_bezdek.txt 3
Iteration 1: SSE = 230.163
Iteration 2: SSE = 81.9806
Iteration 3: SSE = 79.3943
Iteration 4: SSE = 78.9101
Iteration 5: SSE = 78.8514
SW(3) = 0.73566
```

Hint #1: The trace of the within-clusters scatter matrix, $\text{tr}(S_w)$, is the same as SSE, which is already calculated by k -means. Therefore, you do **not** need to calculate $\text{tr}(S_w)$ separately. Also, for $\text{tr}(S_b)$, you do **not** need to calculate the entire S_b matrix. The trace of a square matrix is given by the sum of its diagonal elements. Hence, all you need to do is to calculate the diagonal elements of S_b and then add them up.

Output: Microsoft Excel or Apache OpenOffice Calc table(s) of each internal validity index for various K values for each data set. For each data set and validity index, show the estimated number of clusters in **bold**. **Discuss** which index seems to estimate the number of clusters more accurately and why. For initialization, 4372/5372 students **must** use the “random partition” (Phase 3) method ($R = 100$), whereas 6397 students **must** use the “maximin” (Phase 3) method ($R = 100$, if the method is randomized; $R = 1$, otherwise). All data sets **must** be normalized using the “min-max” (Phase 3) method.

Language: C, C++, or Java. You may **only** use the built-in facilities of these languages. In other words, you may **not** use third-party libraries.

Submission: Submit your source code (C, C++, or Java source files) and output file(s) through Blackboard. Do **not** submit your entire project folder or the input data files I provided to you (I already have them 😊). In addition, do **not** submit your files individually; pack them into a single archive (e.g., zip) and submit the archive file.

Grading: *Functional correctness* and *adherence to good programming practices* are respectively worth **90%** and **10%** of your grade. Given a valid input, a functionally correct program is one that produces the correct output. There are a lot of generic or language-specific guides on good programming practices on the WWW. You **must** identify an appropriate one, include its URL at the top of your program source code(s), and use it throughout this project. In general, you should pay more attention to structural issues (e.g., modularity) than cosmetic ones (e.g., naming and formatting). Remember that if your program is incorrect, whether or not you adhered to good programming practices is immaterial.

Hint #2: There is a Discussion Forum for this phase on Blackboard. As indicated in the syllabus, **frequent** and **meaningful** participation in discussion forums is part of your overall grade. You are strongly encouraged to subscribe to this forum so that you receive email notifications of posts. Remember **not** to post code fragments (or URLs pointing to code fragments) on the forum. In other words, the discussions should be about ideas and algorithms, **not** about their implementation.

Hint #3: Common problems in this phase mainly stem from the careless reading of this document. Read the document **at least** twice before working on your program. Then, read it **at least** once after you are done.